

 hyckomatej Update materials

03c5a68 · 3 months ago 

Name		Name	Last commit date
	..		
	rust-by-example	Update materials	3 months ago
	README.md	Update materials	3 months ago

README.md



2. Lecture - Introduction to Rust

Table of Contents

- [Rust](#)
- [Data Types](#)
- [Variables](#)

[school-of-solana](#) / [2.lesson](#) /

[↑ Top](#)

- [Shadowing](#)
- [Structure](#)
- [Enums](#)
- [Functions](#)
- [Expressions](#)
- [Ownership](#)
- [Borrowing](#)
 - [The Rules of References](#)
 - [In what order will it be dropped?](#)
- [The Slice Type](#)
- [Traits](#)
- [Option<T>](#)
- [Result](#)
- [Generics](#)
- [Macros](#)

Rust

Rust is a modern systems programming language focusing on safety, speed, and concurrency. It accomplishes these goals by being memory safe without using garbage collection.

Rust is a *statically* and *strongly* typed systems programming language. Statically means that all types are known at compile-time, strongly means that these types are designed to make it harder to write incorrect programs.

The big difference from C and C++ is that Rust is *safe by default*. All memory accesses are checked. It is not possible to corrupt memory by accident.

Tip

Learning resources:

- [Rust Book](#)
- [Rust Cheat Sheet](#)

Data Types

Rust is a statically typed language. Every value in Rust is of a certain data type. The compiler can automatically infer data type of the variable based on the value assigned to it.

Use the `let` keyword to declare a variable, or `const` for constants.

Variables

The data type is optional while declaring a variable in Rust. The data type is inferred from the value assigned to the variable.

Mutable

Variables are **immutable by default**. Prefix the variable name with `mut` keyword to make it mutable. The value of a mutable variable can be changed.

Shadowing

Rust allows programmers to declare variables with the same name. In such a case, the new variable **overrides** the previous variable.

Structure

Arrays are used to represent a homogeneous collection of values. Similarly, a structure is another user defined data type available in Rust that allows us to combine data items of different types, including another structure. A structure defines data as a key-value pair.

The `struct` keyword is used to declare a structure. Since structures are statically typed, **every field in the structure must be associated with a data type**.

```
struct Person {  
    first_name: String,
```



```
    last_name: String
}

fn main() {
    let p = Person {
        first_name: "John".to_string(),
        last_name: "Smith".to_string()
    };
    println!("person {} {}", p.first_name,p.last_name);
}
```

Enums

Enums are types which have a few definite values.

```
enum CarType {
    Hatch,
    Sedan,
    SUV
}

fn print_size(car:CarType) {
    match car {
        CarType::Hatch => {
            println!("Small sized car");
        },
        CarType::Sedan => {
            println!("Medium sized car");
        },
        CarType::SUV =>{
            println!("Large sized Sports Utility car");
        }
    }
}
```



```
    }  
  }  
}  
fn main(){  
    print_size(CarType::SUV);  
    print_size(CarType::Hatch);  
    print_size(CarType::Sedan);  
}
```

Functions

We define a function in Rust by entering `fn` followed by a function name and a set of parentheses. We can call any function we've defined by entering its name followed by a set of parentheses. Because `plus_one` is defined in the program, it can be called from inside the `main` function.

```
fn main() {  
    let x = plus_one(5);  
  
    println!("The value of x is: {x}");  
}  
  
fn plus_one(x: i32) -> i32 {  
    x + 1  
}
```



We can define functions to have parameters, which are special variables that are part of a function's signature. When a function has parameters, you can provide it with concrete values for those parameters. The declaration of `plus_one` has one parameter named `x`. The type of `x` is specified as `i32`.

```
(x: i32)
```



Functions can return values. We don't name return values, but we must declare their type after an arrow (`->`). You can return early from a function by using the `return` keyword and specifying a value, but most functions return the last expression implicitly. The return type of `plus_one` is `i32` .

```
-> i32
```



Expressions

In Rust, expressions are fundamental building blocks of the language. Expression is a piece of code that evaluates to a resultant value. Calling a function is an expression. Calling a macro is an expression. A new scope block created with curly brackets is an expression, for example:

```
fn main() {  
    let y = {  
        let x = 3;  
        x + 1  
    };  
  
    println!("The value of y is: {y}");  
}
```



```
{  
    let x = 3;
```



```
x + 1  
}
```

This block evaluates to 4. Expressions do not include ending semicolons. If you add a semicolon to the end of an expression, you turn it into a statement, and it will then not return a value.

Ownership

Ownership is Rust's most unique feature and has deep implications for the rest of the language.

It enables Rust to make memory safety guarantees without needing a garbage collector, so it's important to understand how ownership works.

Ownership is a set of rules that governs how a Rust program manages memory. All programs have to manage the way they use a computer's memory while running. Some languages have garbage collection that constantly looks for no-longer used memory as the program runs; in other languages, the programmer must explicitly allocate and free the memory.

Rust uses a third approach: memory is managed through a system of ownership with a set of rules that the compiler checks. If any of the rules are violated, the program won't compile. None of the features of ownership will slow down your program while it's running.

[Ownership Rules](#)

First, let's take a look at the ownership rules. Keep these rules in mind as we work through the examples that illustrate them:

- Each value in Rust has a variable that's called its *owner*.

- There can only be one owner at a time.
- When the owner goes out of scope, the value will be dropped.

Borrowing

When a function transfers its control over a variable/value to another function temporarily, for a while, it is called borrowing. This is achieved by passing a reference to the variable (eg. `& var_name`) rather than passing the variable/value itself to the function.

The Rules of References

- At any given time, you can have *either* one mutable reference *or* any number of immutable references.
- References must always be valid.

In what order will it be dropped?

```
type Id = u32;

struct Handle(Id);

// ~ destructor
impl Drop for Handle {
    fn drop(&mut self) {
        println!("handle {} dropped!", self.0)
    }
}

fn main() {
```



```
    let handle_0 = Handle(0);
    let handle_2 = create_handle();
    Handle(3);
}

fn create_handle() -> Handle {
    Handle(1);
    Handle(2)
}
```

The Slice Type

A slice is a pointer to a block of memory. Slices can be used to access portions of data stored in contiguous memory blocks. It can be used with data structures like arrays, vectors and strings. Slices use index numbers to access portions of data. The size of a slice is determined at runtime.

Slices let you reference a contiguous sequence of elements in a collection rather than the whole collection. A slice is a kind of reference, so it does not have ownership.

The concepts of ownership, borrowing, and slices ensure memory safety in Rust programs at compile time. The Rust language gives you control over your memory usage in the same way as other systems programming languages, but having the owner of data automatically clean up that data when the owner goes out of scope means you don't have to write and debug extra code to get this control.

Slices give you different *views* of the *same* array:

```
fn main() {
    let ints = [1, 2, 3, 4, 5];
    let slice1 = &ints[0..2];
```



```
let slice2 = &ints[1..]; // open range!

println!("ints {:?}", ints);
println!("slice1 {:?}", slice1);
println!("slice2 {:?}", slice2);
}
```

Traits

Please note that Rust does not spell `struct` `class`. The keyword `class` in other languages is so overloaded with meaning that it effectively shuts down original thinking.

Let's put it like this: Rust structs cannot *inherit* from other structs; they are all unique types. There is no *sub-typing*. They are dumb data.

So how *does* one establish relationships between types? This is where *traits* come in.

```
trait Show {
    fn show(&self) -> String;
}

impl Show for i32 {
    fn show(&self) -> String {
        format!("four-byte signed {}", self)
    }
}

impl Show for f64 {
    fn show(&self) -> String {
        format!("eight-byte float {}", self)
    }
}
```



```

    }
}

fn main() {
    let answer = 42;
    let maybe_pi = 3.14;
    let s1 = answer.show();
    let s2 = maybe_pi.show();
    println!("show {}", s1);
    println!("show {}", s2);
}

```

Option<T>

The `Option<T>` enum has two variants:

- `None` , to indicate failure or lack of value, and
- `Some(value)` , a tuple struct that wraps a `value` with type `T` .

Type `Option` represents an optional value: every `Option` is either `Some` and contains a value, or `None`, and does not. `Option` types are very common in Rust code, as they have a number of uses:

```

fn main() {
    fn might_print(option: Option<&str>) {
        match option {
            Some(text) => println!("The argument contains the following value: '{}'", text),
            None => println!("The argument contains None."),
        }
    }
}

```



```

let something: Option<&str> = Some("Some string is inside");
let nothing: Option<&str> = None;
might_print(something);
might_print(nothing);
}

```

Result

The return is annotated with the Result enum. We specify the types that the Result will contain when the function returns. If the string is long enough, we return a string literal. If there is an error, we will return a message that is a String. This explains the `Result<&str, String>` .

```

fn main() {
    fn check_length(s: &str, min: usize) -> Result<&str, String> {
        if s.chars().count() >= min {
            return Ok(s);
        } else {
            return Err(format!("'{}' is not long enough!", s));
        }
    }

    let func_return = check_length("some string literal", 100);
    let a_str = match func_return {
        Ok(a_str) => a_str,
        Err(error) => panic!("Problem running 'check_length':\n {:?}", error),
    };
    println!("{}", a_str);
}

```



Generics

We use generics to create definitions for items like function signatures or structs, which we can then use with many different concrete data types. To parameterize the types in a new single function, we need to name the type parameter. We'll use `T` because, by convention, type parameter names in Rust are short, often just one letter.

```
fn identity<T>(value: T) -> T {
    value
}

fn main() {
    let x = identity(10);           // x is inferred to be i32
    let y = identity(3.14);        // y is inferred to be f64
    let z = identity("Hello");     // z is inferred to be &str

    println!("x: {}", x);          // prints 10
    println!("y: {}", y);          // prints 3.14
    println!("z: {}", z);          // prints Hello
}
```



The `identity` function accepts a value of type `T` and returns it unchanged.

Macros

Rust provides a powerful macro system that allows meta-programming. As you have seen in the previous example, macros look like functions, except that their name ends with a bang(!), but instead of generating a function call, macros are expanded into source code that gets compiled with the rest of the program. Therefore, they provide more runtime features to a program unlike functions. Macros are an extended version of functions.



```
macro_rules! say_hello {  
    // `()` indicates that the macro takes no argument.  
    () => {  
        // The macro will expand into the contents of this block.  
        println!("Hello!")  
    };  
}
```

Need help?

If you have any questions feel free to reach out to us on [Discord](#).